

Neural network math

Perfect.

Now we go into **mathematical specification mode** — written so an AI agent can implement it directly.

No storytelling. No motivational fluff.

Pure formal definition.



PIECE 3 — COMPLETE NEURAL NETWORK MATHEMATICAL SPECIFICATION

This section defines:

- Model structure
- Parameter shapes
- Forward propagation equations
- Loss function
- Backpropagation equations
- Parameter updates

Everything needed for direct implementation.



1. Model Definition

Input

Each sample:

[
 $X \in \mathbb{R}^{1 \times 6}$
]

Feature vector of size 6.

Output

[
 $Y \in \mathbb{R}^{1 \times 4}$
]

One-hot encoded class label.

Example:

Clap $\rightarrow$ [1, 0, 0, 0]

Knock $\rightarrow$ [0, 1, 0, 0]

2. Network Architecture

Layer sizes:

- Input: 6
 - Hidden Layer 1: 12
 - Hidden Layer 2: 8
 - Output: 4
-

3. Parameter Definitions

Define weight matrices and bias vectors:

Layer 1

[
 $W_1 \in \mathbb{R}^{6 \times 12}$
]

[
 $b_1 \in \mathbb{R}^{1 \times 12}$
]

Layer 2

[
 $W_2 \in \mathbb{R}^{12 \times 8}$
]

[
 $b_2 \in \mathbb{R}^{1 \times 8}$
]

Output Layer

[
 $W_3 \in \mathbb{R}^{8 \times 4}$
]

[
 $b_3 \in \mathbb{R}^{1 \times 4}$
]

4. Initialization

Initialize weights using small random values:

```
[  
W_i \sim \mathcal{N}(0, \sigma^2)  
]
```

Recommended:

```
[  
\sigma = \sqrt{\frac{2}{n_{\text{input}}}}  
]
```

(He initialization for ReLU)

Biases initialized to zero.

5. Forward Propagation

Step 1 — Layer 1

```
[  
Z_1 = X W_1 + b_1  
]
```

```
[  
A_1 = \text{ReLU}(Z_1)  
]
```

Where:

```
[  
\text{ReLU}(x) = \max(0, x)  
]
```

Step 2 — Layer 2

```
[  
Z_2 = A_1 W_2 + b_2  
]
```

```
[  
A_2 = \text{ReLU}(Z_2)  
]
```

Step 3 — Output Layer

```
[  
Z_3 = A_2 W_3 + b_3  
]
```

```
[
A_3 = Softmax(Z_3)
]
```

Where:

```
[
Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^4 e^{z_j}}
]
```

Output:

```
[
\hat{Y} = A_3
]
```

6. Loss Function

Use Categorical Cross Entropy:

```
[
L = -\sum_{i=1}^4 Y_i \log(\hat{Y}_i)
]
```

For numerical stability:

- Clip probabilities to small epsilon (e.g., 1e-9).
-

7. Backpropagation

We compute gradients layer by layer.

◆ Step 1 — Output Layer Gradient

Because Softmax + Cross Entropy:

```
[
dZ_3 = \hat{Y} - Y
]
```

```
[
dW_3 = A_2^T dZ_3
]
```

```
[
db_3 = dZ_3
]
```

◆ Step 2 — Hidden Layer 2

$$\begin{bmatrix} dA_2 = dZ_3 W_3^T \end{bmatrix}$$

Derivative of ReLU:

$$\begin{bmatrix} \text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \end{bmatrix}$$

$$\begin{bmatrix} dZ_2 = dA_2 \odot \text{ReLU}'(Z_2) \end{bmatrix}$$

$$\begin{bmatrix} dW_2 = A_1^T dZ_2 \end{bmatrix}$$

$$\begin{bmatrix} db_2 = dZ_2 \end{bmatrix}$$

◆ Step 3 — Hidden Layer 1

$$\begin{bmatrix} dA_1 = dZ_2 W_2^T \end{bmatrix}$$

$$\begin{bmatrix} dZ_1 = dA_1 \odot \text{ReLU}'(Z_1) \end{bmatrix}$$

$$\begin{bmatrix} dW_1 = X^T dZ_1 \end{bmatrix}$$

$$\begin{bmatrix} db_1 = dZ_1 \end{bmatrix}$$

8. Parameter Update Rule

Use Gradient Descent:

```
[  
W_i = W_i - \alpha dW_i  
]
```

```
[  
b_i = b_i - \alpha db_i  
]
```

Where:

```
[  
\alpha = \text{learning rate}  
]
```

Recommended initial value:

```
[  
\alpha = 0.01  
]
```

9. Training Loop Definition

For each epoch:

For each training sample:

1. Forward pass
2. Compute loss
3. Backprop
4. Update weights

After epoch:

- Compute validation accuracy

Stop after fixed epochs (e.g., 200–500)
or early stopping if validation loss increases.

10. Prediction Rule

Given output vector:

```
[  
\hat{Y}  
]
```

Predicted class:

[
 $\text{argmax}(\hat{Y})$
]

11. Accuracy Formula

[
 $\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Samples}}$
]

12. Numerical Stability Requirements

- Use log-softmax trick if possible
 - Clip softmax outputs to avoid $\log(0)$
 - Use double precision
-

What This Defines

Your AI agent can now implement:

- Parameter allocation
- Forward pass
- Backprop
- Weight updates
- Training loop
- Prediction

Without asking you conceptual questions.

Now I will test whether you truly understand.